

PYTHON

Module 14: List Mastery & Collections

CODING

Mastering Python Lists

Lists are the workhorse of Python collections. They are used to store multiple items in a single, ordered variable.

Key Characteristics at CodeHive:

- **Ordered:** Items maintain a defined sequence.
- **Changeable:** You can modify, add, or remove items after creation.
- **Duplicates Allowed:** Since items are indexed, multiple identical values can coexist.

Lists are created using square brackets `[]` or the `list()` constructor.

Accessing & Slicing Data

Accessing data efficiently is vital for large datasets.

• **Indexing:** Use `list[0]` for the first item. Negative indexing (`list[-1]`) targets the end of the list.

• **Range of Indexes:** `list[2:5]` returns a sub-list from index 2 to 4.

- Check Existence: Use the `in` keyword to search for an item within the list instantly.

Modifying List Content

Lists are mutable, meaning they can be reshaped on the fly.

- Direct Assignment:

`thislist[1] = "blackcurrant"` replaces a specific item.

- Range Replacement: You can replace a slice with a new list of any length: `thislist[1:3] = ["A", "B", "C"]` .

- Insertion: Use `.insert(index, value)` to add an item without overwriting existing data.

Adding & Extending Lists

There are three primary ways to grow a list:

- `append()` : Adds a single element to the very end.

- `insert()` : Places an item at a specific coordinate.

- `extend()` : Merges another iterable (list, tuple, or set) into the current list.

At CodeHive, we prefer `extend()` over loops for joining collections to improve performance.

Removing & Clearing Data

Maintaining clean data requires robust removal tools:

- `remove("val")` : Deletes the first occurrence of a specific value.

- `pop(index)` : Removes and returns the item at a specific index. If no index is provided, it pops the last item.

- `del` : A keyword used to delete a specific index or wipe the entire variable from memory.

•

`clear()` : Empties the list but keeps the variable structure intact.

Iteration & Looping

Processing list items is done through various loops:

• For Loop: The standard way to iterate directly over items.

• Index Loop: Using `range(len(thislist))` to iterate using coordinates.

• While Loop: Manual control over the index increment.

• Short-hand: `[print(x) for x in list]` is a one-line iteration method.

List Comprehension

This is one of Python's most powerful features. It offers a shorter syntax for creating new lists based on existing ones.

Formula: `newlist = [expression for item in iterable if condition]`

Example: `[x.upper() for x in fruits if "a" in x]`

This single line replaces multiple lines of code, making your CodeHive projects more elegant and faster.

Sorting Strategies

Lists can be organized numerically or alphabetically using `.sort()`.

Advanced Sorting:

- Descending: Use `.sort(reverse = True)`.

- Custom Keys: Use `key = function` to sort by specific logic (e.g., length of strings or absolute values).

- Case Sensitivity: Use `key = str.lower` for a true alphabetical sort that ignores casing.

Copying & Joining

Warning: You cannot copy a list with `list2 = list1` . This only creates a reference to the same memory object.

Correct Copy Methods:

- `list.copy()` method.

- `list(existing_list)` constructor.

- Slicing: `mylist = thislist[:]` .

Joining: Use the `+` operator for simple concatenation or `extend()` for professional workflows.

List Method Reference

A summary of essential tools for the CodeHive developer:

- `count()` : Returns how many times a value appears.

- `reverse()` : Flips the order of the list.

- `index()` : Finds the first occurrence of a value.

Mastering these methods is the final step in becoming a Python List expert.